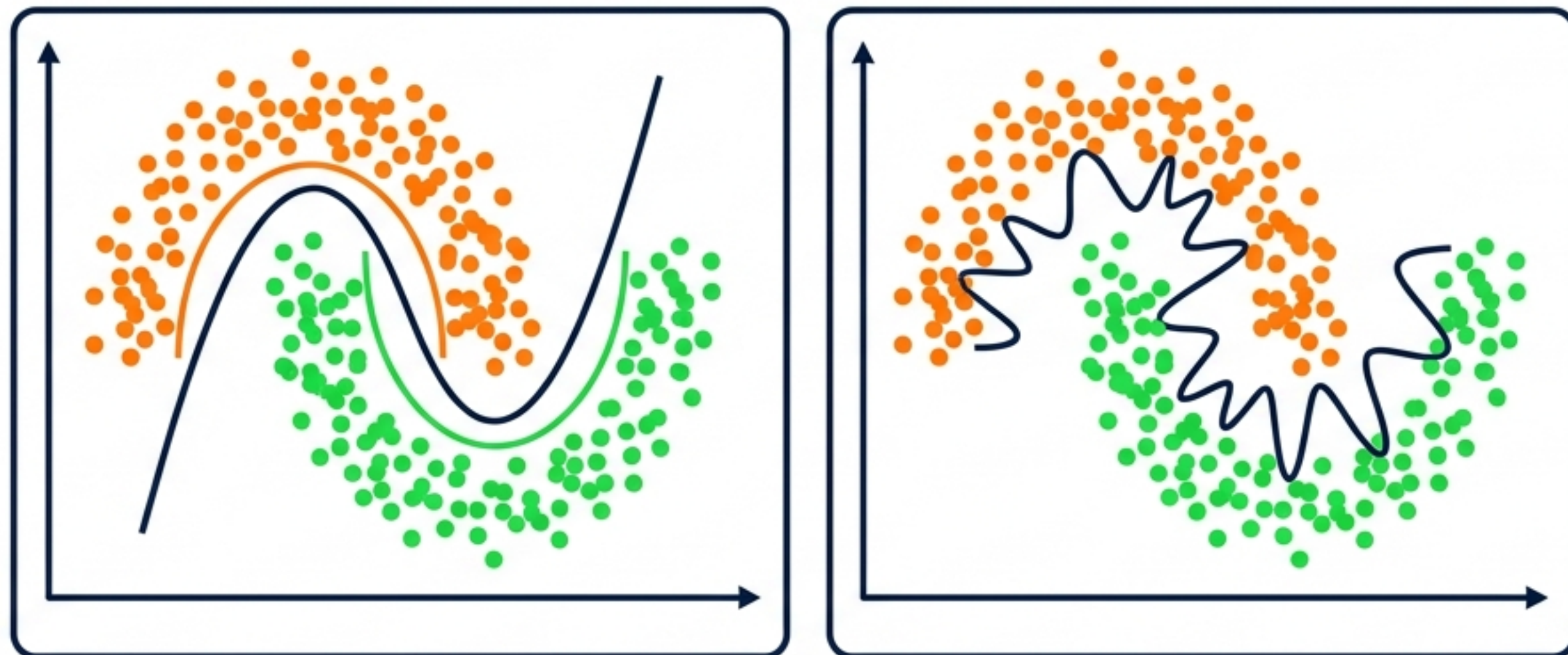


2.5 Kernel Đa thức (Polynomial Kernel)



```
SVC(kernel='poly', degree=3)
```



Hình 5-7: Polynomial Kernel với bậc 3 và 10

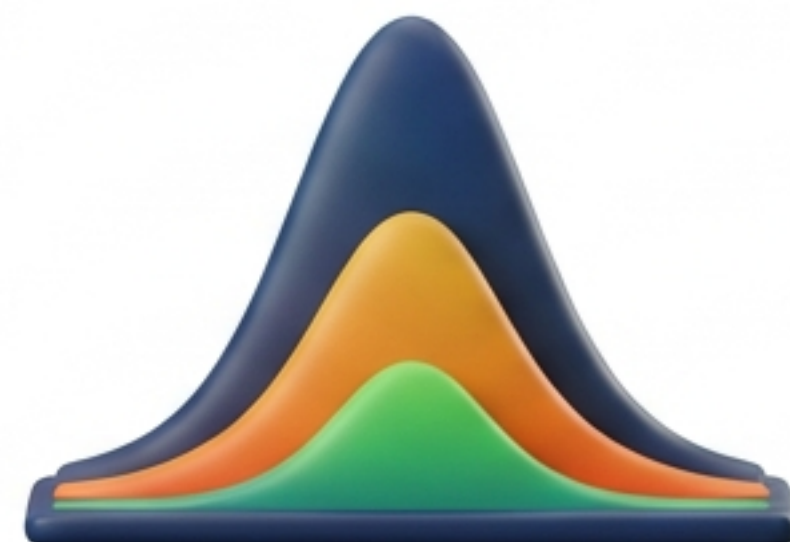
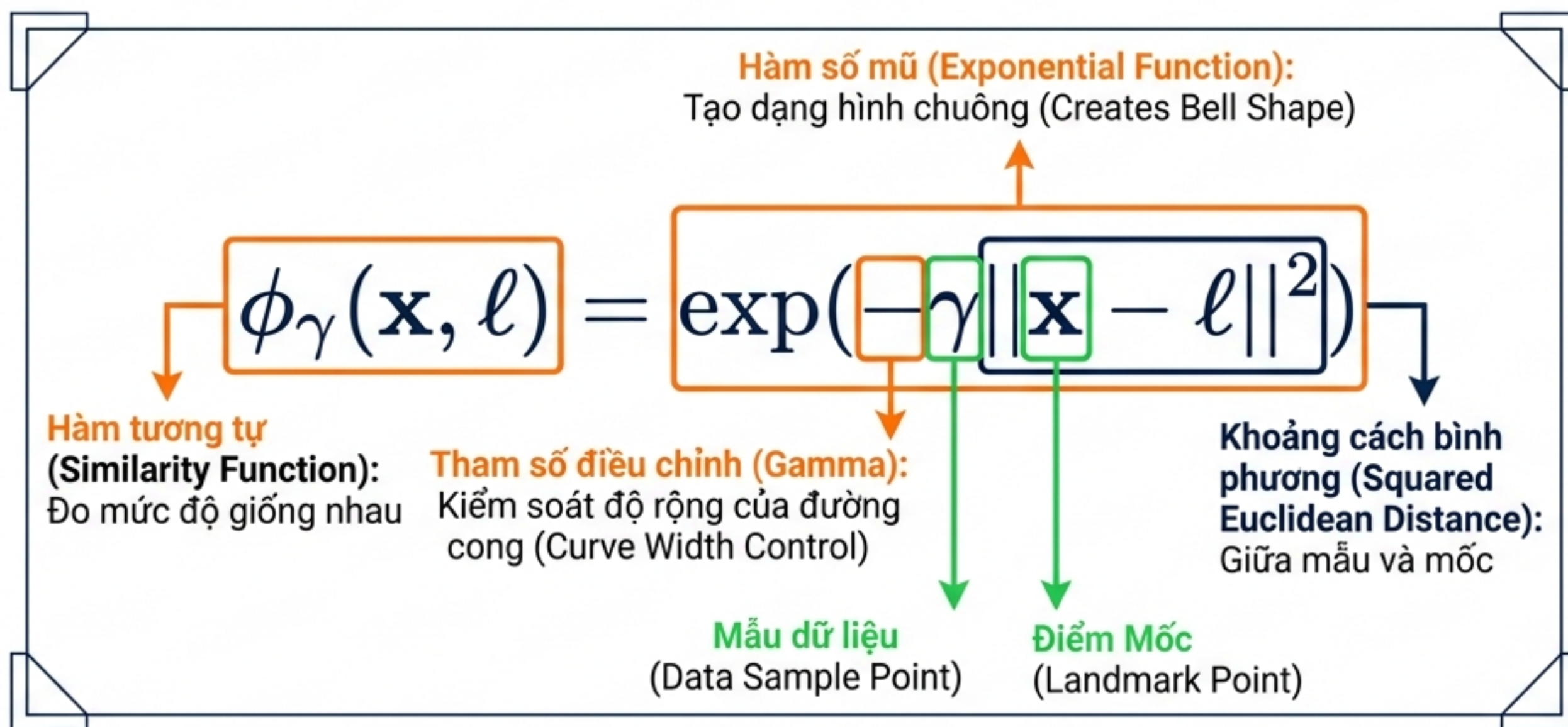
Tham số `coef0`: Kiểm soát mức độ ảnh hưởng của đa thức bậc cao so với đa thức bậc thấp.

Insight: Hình ảnh cho thấy sự khác biệt giữa mô hình bậc 3 (trái) và bậc 10 (phải).

2.6 Đặc trưng tương tự (Similarity Features)

Kỹ thuật bổ sung: Sử dụng **Hàm tương tự (Similarity Function)**, đo lường mỗi mẫu giống các **Mốc (Landmarks)** đến mức nào.

Ví dụ: Gaussian Radial Basis Function (RBF).



Đặc điểm: Giá trị dạng hình chuông. Thay đổi từ 0 (rất xa) đến 1 (ngay tại mốc).

2.7 Minh họa Hàm Gaussian RBF

1

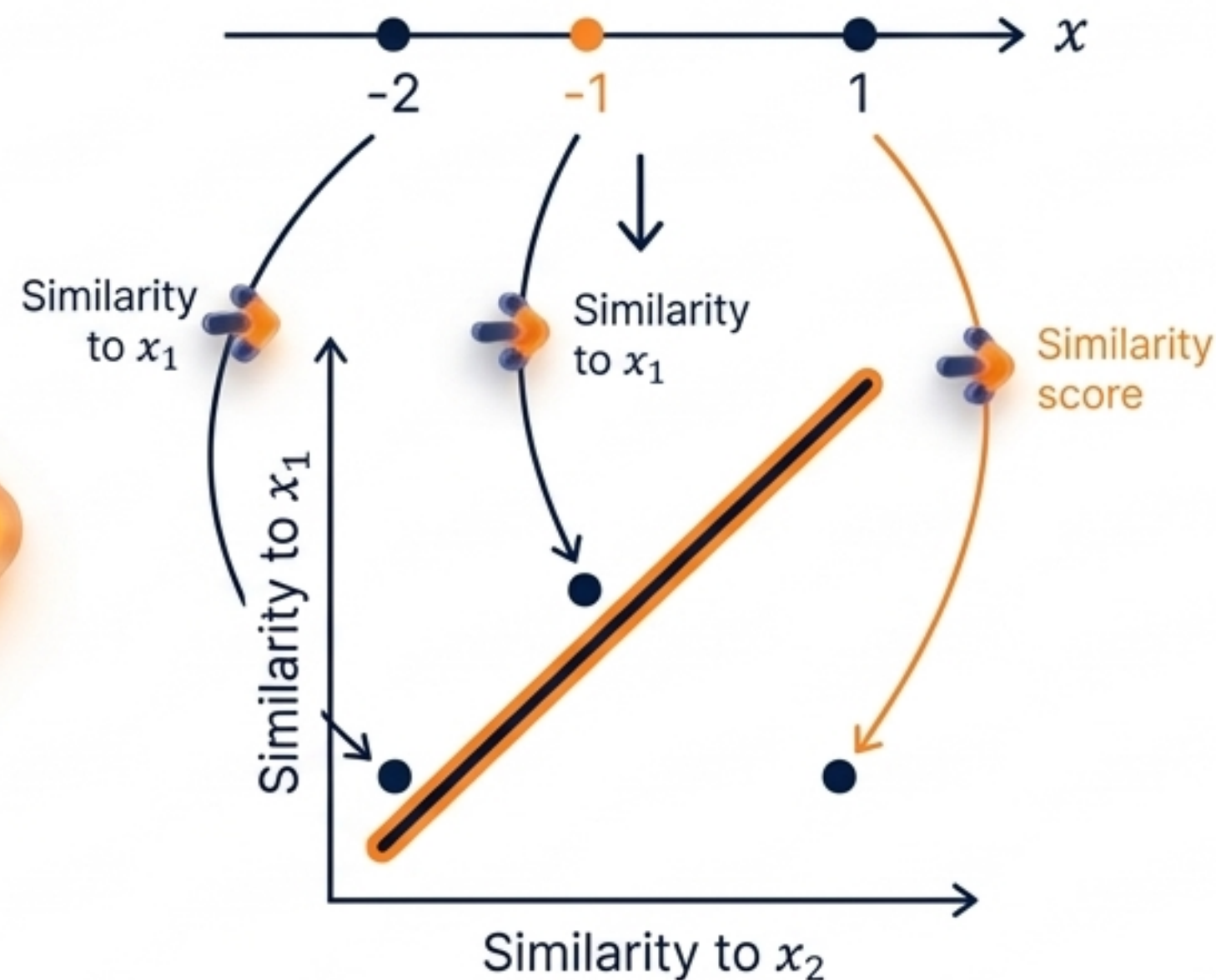
Bước 1: Đặt mốc tại $x_1 = -2$ và $x_2 = 1$.

2

Bước 2: Chuyển điểm $x = -1$ thành 2 chiều mới: khoảng cách tới mốc 1 và khoảng cách tới mốc 2.

3

Kết quả: Giờ đây tập dữ liệu đã có thể phân tách tuyến tính bằng 1 đường thẳng.



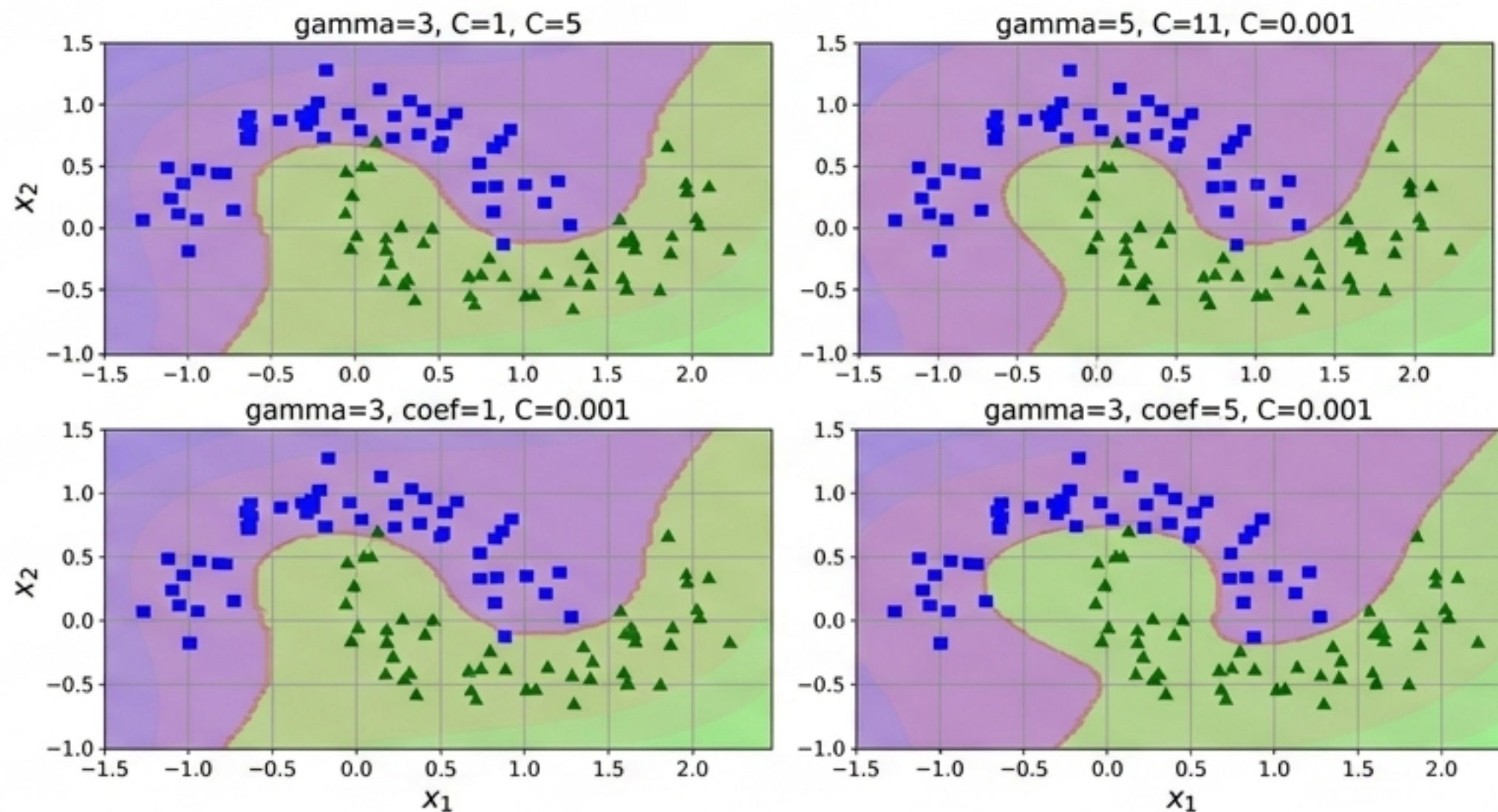
Hình 5-8: Sử dụng Similarity Features phân tách dữ liệu 1D

2.8 Gaussian RBF Kernel

Lợi ích: Giống đa thức, dùng Kernel RBF để hưởng lợi ích của vô vàn các Mốc (Landmarks) mà không phải tính toán chi phí cao.

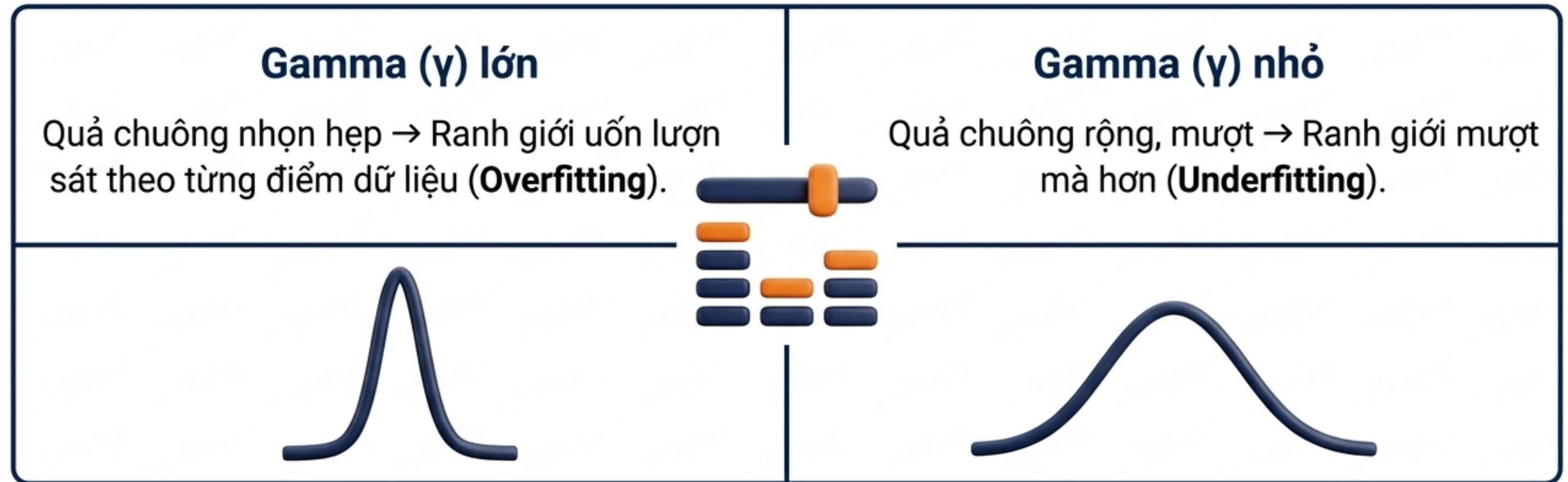
```
SVC(kernel='rbf', gamma=5, C=0.001)
```

Vai trò của Gamma: Tham số gamma (γ) định hình độ hẹp/rộng của quả chuông.



Hình 5-9: Hiệu ứng của γ và C lên SVM Kernel RBF

2.9 Điều chỉnh siêu tham số Gamma và C



 **Chiến lược** Nếu mô hình bị Overfitting, hãy giảm γ . Ngược lại, hãy tăng γ . (Tương tự như C).

 **Rule of thumb** Khi nghi ngờ, hãy luôn thử Linear Kernel trước, sau đó tới Gaussian RBF Kernel.

3.1 Các Lớp của Scikit-Learn cho SVM



LinearSVC	SVC	SGDClassifier(loss='hinge')
- Dùng thư viện LIBLINEAR. - Tối ưu cho dữ liệu cực lớn. - Phức tạp thời gian: $O(m \times n)$ (Chỉ chạy tuyến tính,	- Dùng thư viện LIBSVM (hỗ trợ Kernel trick). - Phức tạp: $O(m^2 \times n)$ đến $O(m^3 \times n)$ - Quá chậm nếu $> 100,000$ mẫu (phần “Quá chậm”)	- Huấn luyện Gradient Descent ngẫu nhiên. - Hữu ích cho tập dữ liệu khổng lồ (Out-of-core learning)

3.2 Ý tưởng Hồi quy bằng SVM

Đa năng: Thuật toán SVM không chỉ phân loại tốt mà còn hồi quy (dự đoán giá trị số liên tục) xuất sắc!

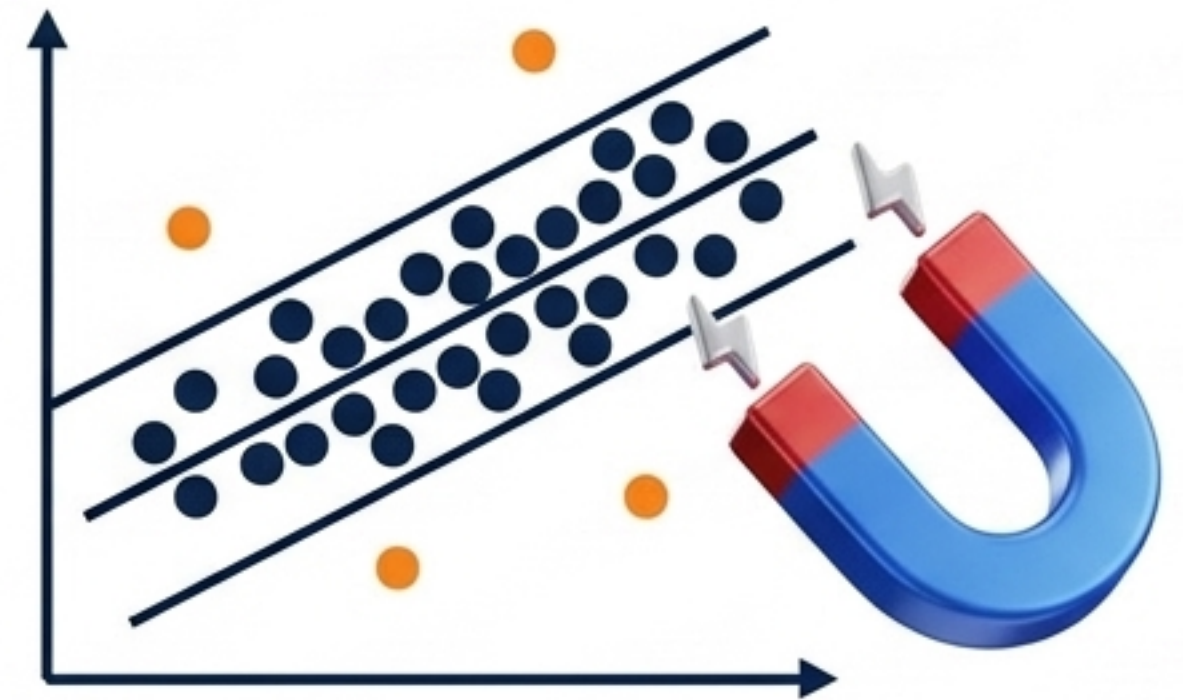
Phân loại SVM (Trái)

Mục tiêu là dải lề rộng nhất nhưng KHÔNG chứa bất cứ điểm vi phạm nào bên trong.



Hồi quy SVM (Phải)

Mục tiêu là đưa CÀNG NHIỀU ĐIỂM VÀO BÊN TRONG dải lề càng tốt, hạn chế điểm rơi ra ngoài.



Kiểm soát: Độ rộng lề được kiểm soát bởi siêu tham số Epsilon (ϵ).

3.3 Minh họa SVM Regression Tuyến tính

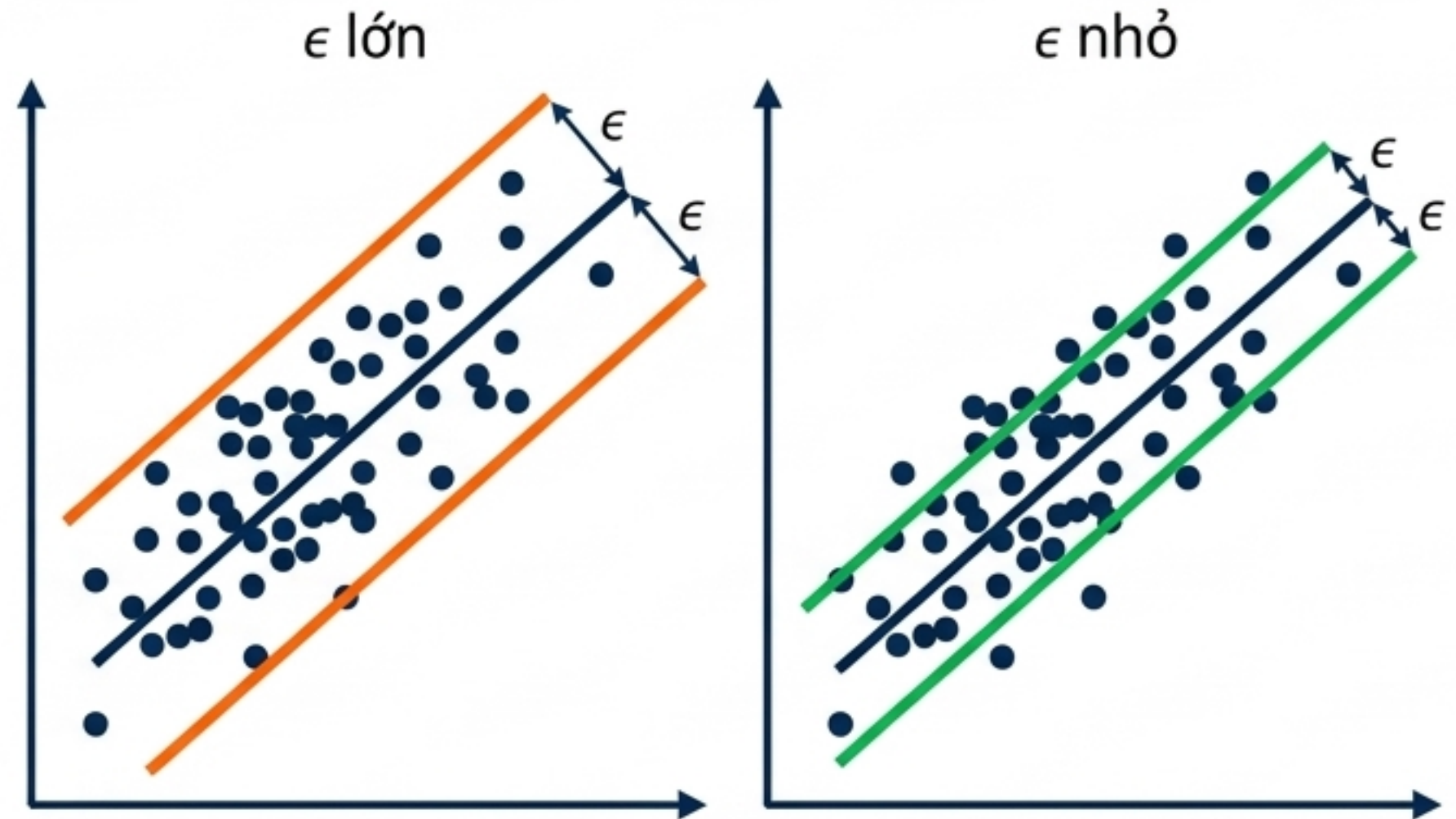


Lớp: LinearSVR.

Đặc tính (ϵ -insensitive):
Việc thêm nhiều điểm dữ liệu nằm bên trong biên không làm ảnh hưởng đến dự đoán của mô hình.

So sánh:

ϵ lớn (trái): Lề rộng.
| ϵ nhỏ (phải): Lề hẹp.



Hình 5-10: Hồi quy LinearSVR với các giá trị ϵ khác nhau

3.4 Hồi quy SVM Phi tuyến (Nonlinear SVR)

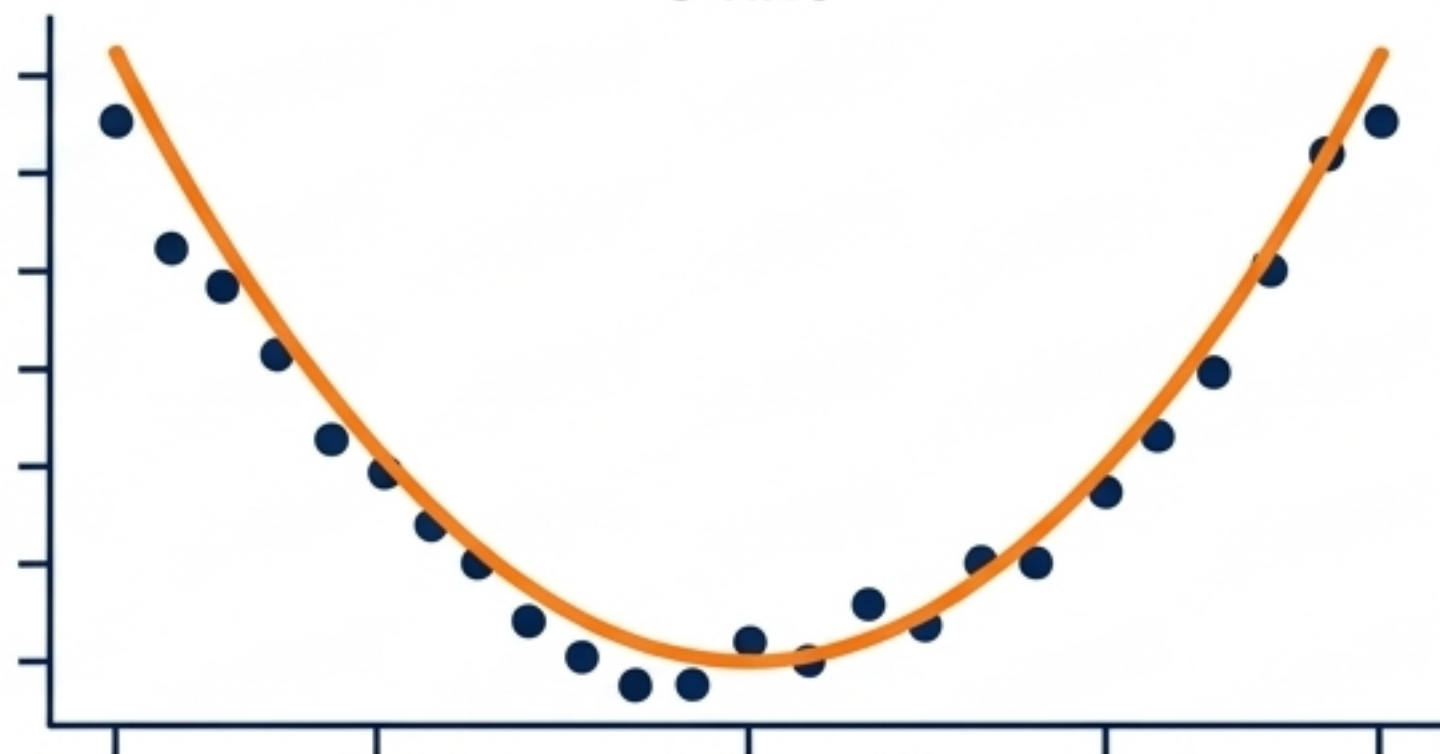


Kernel Trick: Hỗ trợ thông qua lớp **SVR(kernel='poly')**. (Dữ liệu trong hình theo dạng hàm bậc 2, bám sát xu hướng với đường cong mịn màng).

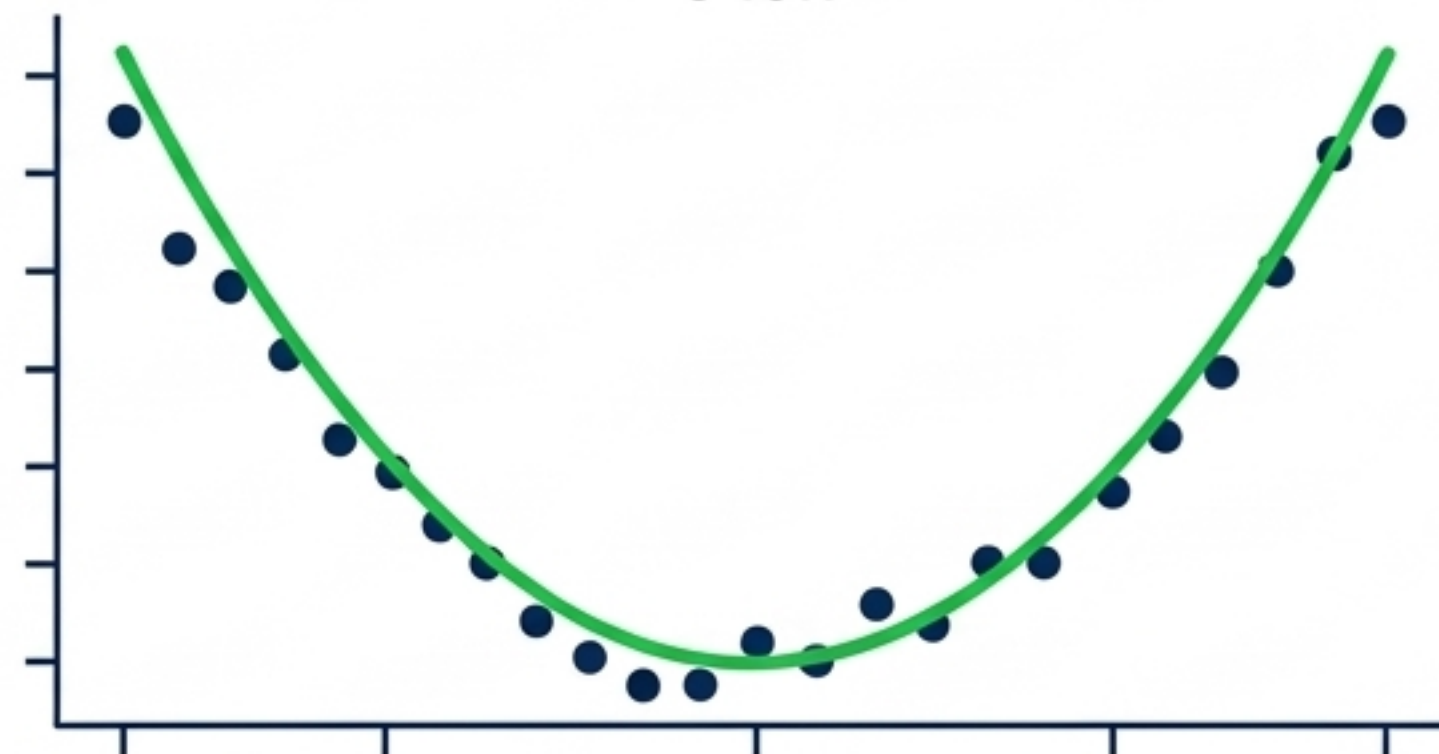
Ảnh hưởng của C:

C nhỏ (trái): Mô hình tự do, linh hoạt. | C lớn (phải): Ít sai số, sát dữ liệu hơn (ít regularization).

C nhỏ



C lớn



Hình 5-11: Hồi quy SVM phi tuyến bằng Kernel Đa thức

4.1 Hàm Quyết định (Decision Function)

$$\sum f(\mathbf{x}) =$$

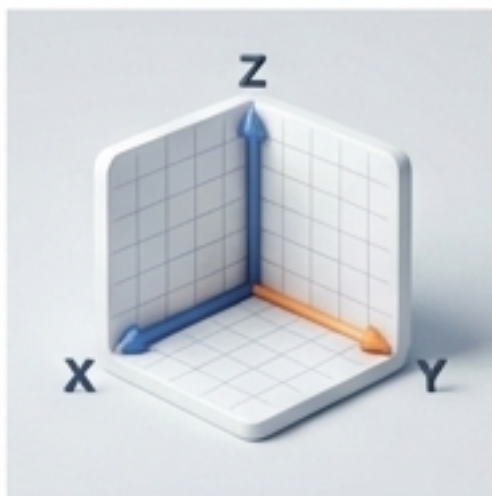
Nguyên lý: SVM **tuyến tính** dự đoán bằng cách tính hàm quyết định.

$$f(\mathbf{x}) = \mathbf{w}^T \cdot \mathbf{x} + b = w_1 x_1 + \dots + w_n x_n + b$$

Quy tắc dự đoán:

Nếu kết quả dương $\rightarrow$ dự đoán lớp 1 (Tích cực).	Nếu âm $\rightarrow$ dự đoán lớp 0 (Tiêu cực).
$\hat{y} = 1$ if $\mathbf{w}^T \mathbf{x} + b \geq 0$, else 0.	

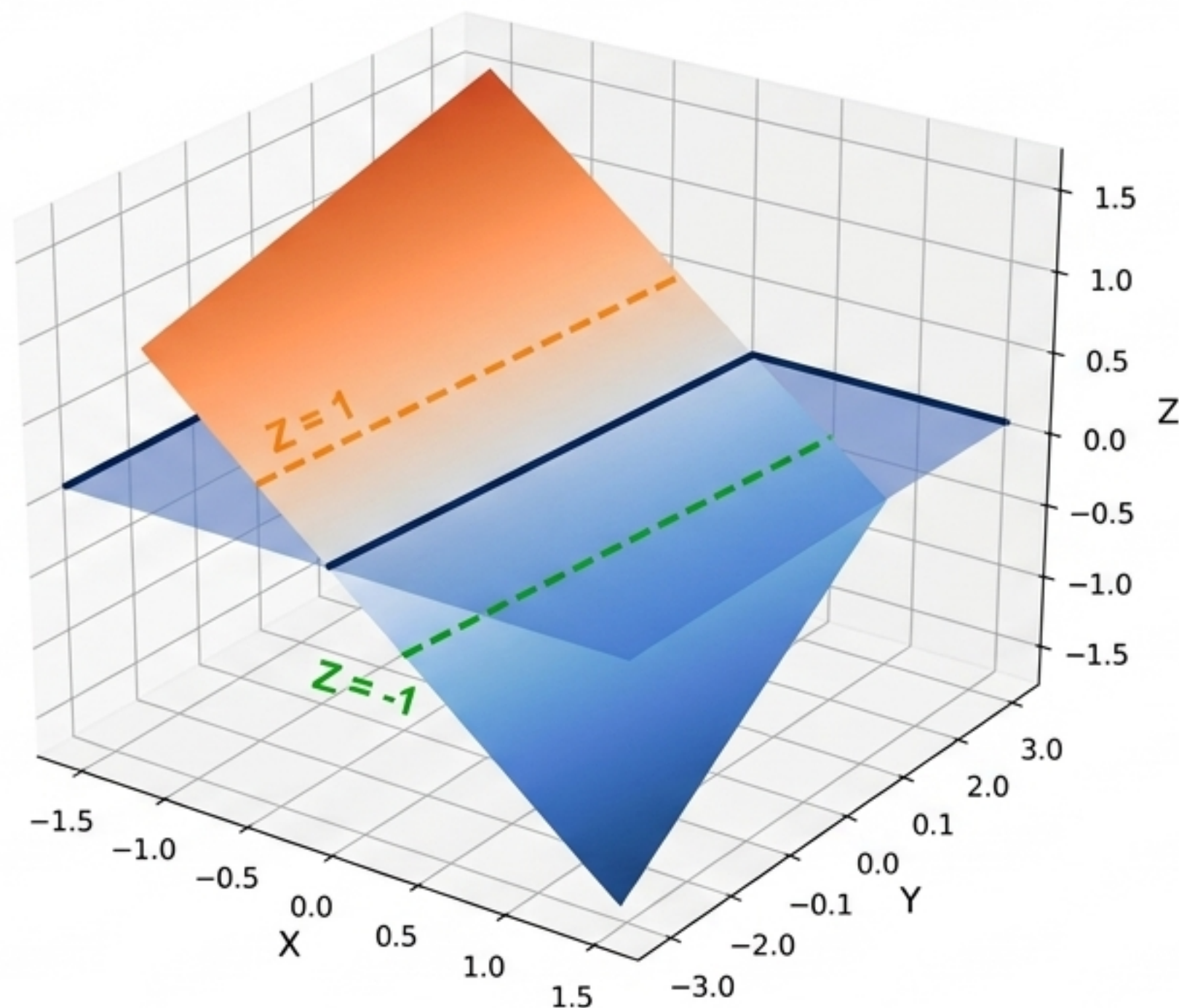
4.2 Đồ thị Không gian Hàm quyết định



Không gian 3 chiều: Trục Z là giá trị của hàm quyết định.

Ranh giới (Decision Boundary): Mặt phẳng cắt tại $Z = 0$ tạo thành ranh giới quyết định (Đường liền đen).

Đường lề (Margins): Đường lề đứt nét tương ứng với mặt cắt $Z = 1$ và $Z = -1$.



Hình 5-12: Decision function cho mô hình Iris Dataset 2D

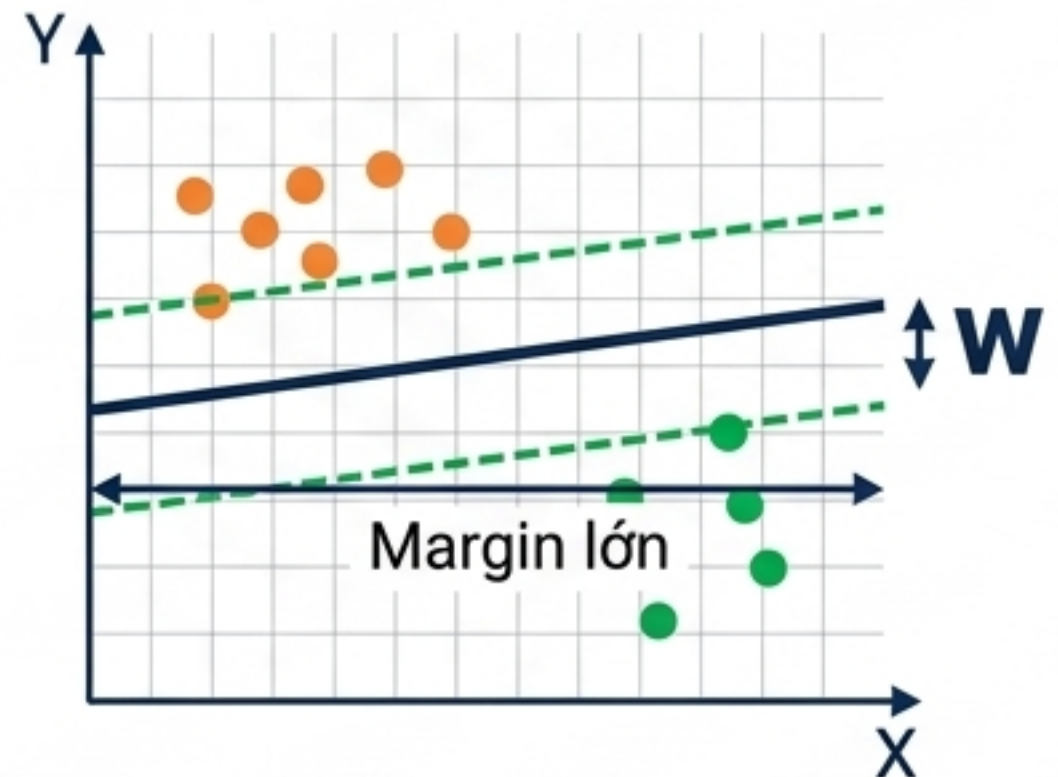
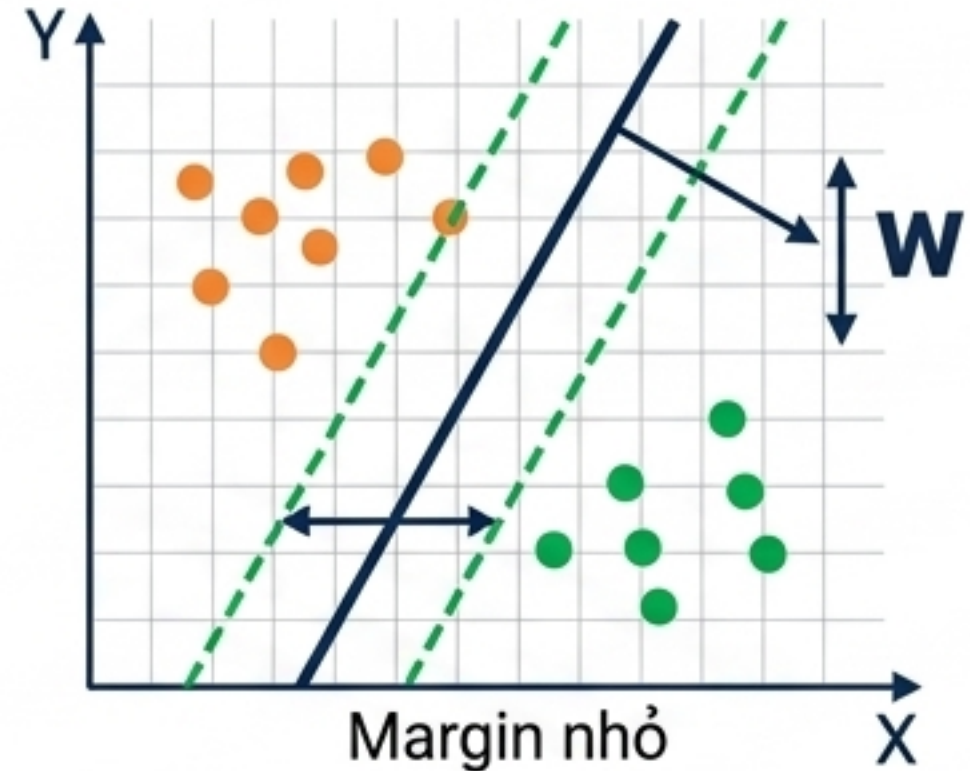
4.3 Mối liên hệ giữa Vector Trọng số (w) và Biên



Cơ chế: Kích thước của véc-tơ trọng số w xác định độ dốc của mặt phẳng hàm.

Nghịch đảo: Nếu chia đôi độ dốc w , độ rộng của lề (Margin) sẽ nhân đôi!

Mục tiêu: Để có Margin cực đại, ta phải tối thiểu hóa độ lớn của w , tức là $||w||$.



Hình 5-13: Trọng số w càng nhỏ, Lề margin càng lớn

4.4 Hàm Mục Tiêu của Phân loại Biên Cứng



Mục tiêu & Quy ước

- **Mục tiêu chính:** Tối thiểu hóa $\|\mathbf{w}\|$ (để lề rộng nhất).
- Mẫu lớp 1: $f(\mathbf{x}) \geq 1$
- Mẫu lớp 0: $f(\mathbf{x}) \leq -1$
- **Quy ước:** Đặt $t^{(i)} = 1$ nếu $y^{(i)} = 1$ và $t^{(i)} = -1$ nếu $y^{(i)} = 0$.
- **Gộp lại:** $t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1$ với mọi mẫu.

Bài toán tối ưu hóa (Center Block)

$$\text{Cực tiểu hóa: } \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

$$\text{Sao cho: } t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad \forall i$$

4.5 Hàm Mục Tiêu của Phân loại Biên Mềm

Khái niệm Biên Mềm: Chấp nhận lỗi.
Định nghĩa biến Slack $\zeta(i) \geq 0$, đo lường mức độ mẫu thứ i được phép vi phạm lề.



Mâu thuẫn: Tối thiểu hóa 2 mục tiêu:
làm cho $\mathbf{w}$ nhỏ nhất vs làm cho tổng ζ nhỏ nhất.



$$\text{Cực tiểu hóa: } \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^m \zeta(i)$$

Vai trò của C: Siêu tham số C xác định sự đánh đổi (trade-off) giữa hai mục tiêu này.

4.6 Trình giải Bài toán (Solvers)

Bản chất bài toán: Đây là các bài toán Quy hoạch Toàn phương (Quadratic Programming - QP) có lời giải tối ưu toàn cục.

Các Solvers phổ biến

- ✓ SMO
- ✓ LIBLINEAR
- ✓ LIBSVM



Cách tiếp cận thay thế

Dùng Gradient Descent thông thường để cực tiểu hóa Hinge Loss. Đó chính là cách hoạt động của lớp **SGDClassifier**.